

PROPLEDGER ARCHITECTURE SERIES — VOLUME 10 OF 10

Full-Stack Mastery Handbook

End-to-End System Integration, Testing & Executive Interview Playbook

DOMAIN LEVEL:

Enterprise Real Estate SaaS

Comparable System: Yardi Voyager / RealPage

FRAMEWORKS:

Spring Boot 3.3 | PG 16 | React 19

PAGE BUDGET:

10 Pages Complete Reference

1. DOCUMENT EXECUTIVE ABSTRACT & SCOPE

This final volume synthesizes the entire PropLedge platform into an executive-level interview and system integration master handbook. Covering automated testing pyramids, disaster recovery protocols, database partitioning, and a 360-degree interview presentation framework, it empowers candidates to demonstrate mastery of enterprise software engineering.

2. VOLUME SYLLABUS & CORE CAPABILITIES

Page	Chapter Title	Primary Technical Focus & Learning Outcome
02	Full-Stack System Integration	Axios request interceptors, Bearer token injection, and 401 handling
03	CORS Preflight Handshakes	OPTIONS handshakes, allow-headers, and cross-origin security policies
04	Automated Testing Strategy	JUnit 5, Mockito, SpringBootTest, Testcontainers, and Vitest testing
05	Flyway Migration Discipline	Baseline version control, repeatable scripts, and checksum integrity
06	Disaster Recovery & WAL Logs	Point-In-Time Recovery (PITR), pg_dump, and replication stream safety
07	Enterprise Scalability Models	Declarative table partitioning, sharding strategies, and Redis caching
08	360-Degree Pitch Guide	How to present PropLedge to senior real estate tech hiring managers
09	System Design Leadership	Tradeoff articulation, engineering decisions, and failure mode analysis
10	Master Technical Cheat Sheet	Essential commands, ports, URLs, credentials, and repository index

SENIOR STAFF ENGINEER REVIEWER GUIDANCE

This volume is engineered specifically for software engineering and database architecture interviews at Tier-1 real estate technology and enterprise SaaS corporations (Yardi, RealPage, AppFolio, CoStar). Every code sample reflects active, compilable implementations from the PropLedge production repository.

2. FULL-STACK SYSTEM INTEGRATION & NETWORK TRANSPORT

AXIOS INTERCEPTORS, BEARER TOKEN INJECTION & ERROR HANDLING

1. The Centralized API Transport Layer

In enterprise frontends, network calls are never scattered with raw `fetch()` statements. PropLedge establishes a centralized Axios transport client with automated Bearer token injection and global 401 response interceptors:

```
// CODE LISTING: TYPESCRIPT - CENTRALIZED AXIOS CLIENT IMPLEMENTATION

// propledger-frontend/src/services/api.ts
import axios from 'axios';

const api = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL || 'http://localhost:8080',
  headers: { 'Content-Type': 'application/json' },
});

// REQUEST INTERCEPTOR: Inject Bearer Token
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// RESPONSE INTERCEPTOR: Handle 401 Unauthorized Globally
api.interceptors.response.use(
  (response) => response,
```

3. CROSS-ORIGIN RESOURCE SHARING (CORS) SECURITY

PREFLIGHT OPTIONS HANDSHAKES, ACCESS HEADERS & SECURITY

1. The Anatomy of a CORS Preflight Request

When the browser sends an API request with custom headers (Authorization: Bearer) or Content-Type: application/json, it dispatches an HTTP OPTIONS request before the real request:

- 1. Client OPTIONS Request: Browser sends 'Origin: http://localhost:5173' and 'Access-Control-Request-Headers: authorization, content-type'.
- 2. Spring Boot Response: Spring Security returns 200 OK with 'Access-Control-Allow-Origin: http://localhost:5173' and 'Access-Control-Max-Age: 3600'.
- 3. Real Request Execution: Browser caches permission for 1 hour and immediately dispatches the actual POST /api/payments request.

2. CORS Configuration Source in SecurityConfig.java

// CODE LISTING: JAVA 21 – CORS CONFIGURATION SOURCE

```
@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration configuration = new CorsConfiguration();
    configuration.setAllowedOrigins(List.of("http://localhost:5173", "http://localhost:3000"));
    configuration.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
    configuration.setAllowedHeaders(List.of("Authorization", "Content-Type", "Accept"));
    configuration.setAllowCredentials(true);
    configuration.setMaxAge(3600L); // Cache preflight for 1 hour

    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", configuration);
    return source;
}
```

4. ENTERPRISE AUTOMATED TESTING PYRAMID

UNIT TESTS, INTEGRATION MOCKS & TESTCONTAINERS DISCIPLINE

1. The Enterprise Testing Pyramid

Test Layer	Framework / Technology	Execution Speed	Testing Scope
Unit Tests	JUnit 5 + Mockito	< 50ms per test	Service business logic, validations, mappers.
Slice Tests	Spring @WebMvcTest	~300ms per test	Controller status codes, JSON serialization, security.
Integration Tests	SpringBootTest + Testcontainers	~2s per test	Real PostgreSQL 16 container, triggers, constraints.
Frontend Tests	Vitest + React Testing Library	< 100ms per test	Component rendering, user events, form validation.

2. Production Code: Mockito Unit Test

```
// CODE LISTING: JAVA 21 – MOCKITO UNIT TEST IMPLEMENTATION

@ExtendWith(MockitoExtension.class)
class LeaseServiceTest {
    @Mock private LeaseRepository leaseRepository;
    @Mock private UnitRepository unitRepository;
    @InjectMocks private LeaseServiceImpl leaseService;

    @Test
    void createLease_WhenUnitAlreadyOccupied_ThrowsBusinessException() {
        Unit unit = Unit.builder().id(1L).status(UnitStatus.OCCUPIED).build();
        when(unitRepository.findById(1L)).thenReturn(Optional.of(unit));

        assertThrows(BusinessRuleException.class, () ->
            leaseService.createLease(LeaseCreateDto.builder().unitId(1L).build())
        );
    }
}
```

5. FLYWAY MIGRATION DISCIPLINE & VERSION CONTROL

DETERMINISTIC SCHEMA BASELINES & CHECKSUM VERIFICATION

1. Database Migration Best Practices

- 1. Immutability: Once a migration script (e.g. V5__create_leases.sql) is merged, it is NEVER modified in-place. Changes require a new V13 migration.
- 2. Checksum Verification: Flyway calculates a SHA-256 checksum of every migration file in 'flyway_schema_history'. If a file is altered, startup fails immediately.
- 3. Idempotent Rollbacks: DDL scripts use IF EXISTS / IF NOT EXISTS where appropriate and avoid dropping production data.

2. Migration File Inventory

Migration Script	Key Tables Created	Domain Purpose
V1__create_roles_users.sql	roles, users, user_roles	Identity, authentication, and RBAC.
V2__create_properties.sql	owners, properties	Real estate asset hierarchy.
V3__create_buildings_units.sql	buildings, units	Rentable structures and spaces.
V5__create_leases.sql	leases, lease_tenants	Contracts and btree_gist temporal exclusion.
V6__create_financial_tables.sql	invoices, items, payments, allocs	Accounts Receivable & settlement subledger.
V12__create_triggers.sql	Triggers & Functions	ACID balance maintenance and status sync.

6. DISASTER RECOVERY & POINT-IN-TIME RECOVERY (PITR)

WRITE-AHEAD LOG ARCHIVING & ZERO-DATA-LOSS BACKUP PROTOCOLS

1. Point-In-Time Recovery (PITR) Architecture

In enterprise financial platforms, standard nightly dumps are insufficient (losing up to 24 hours of transactions). PropLedger mandates Point-In-Time Recovery (PITR):

- 1. Base Backups: Weekly physical file system backups taken via `pg_basebackup`.
- 2. Continuous WAL Archiving: Every 16MB Write-Ahead Log segment is continuously streamed to an S3 bucket via `archive_command`:
`archive_command = 'test ! -f /mnt/wal_archive/%f && cp %p /mnt/wal_archive/%f'`
- 3. Microsecond Recovery: To restore after a catastrophic event at 14:32:10 UTC, the DBA restores the base backup and replays WAL segments up to `target_time = '2026-09-12 14:32:09'`, recovering to the exact second before failure!

7. ENTERPRISE SCALABILITY: PARTITIONING & SHARDING

HANDLING BILLIONS IN TRANSACTION VOLUME ACROSS LARGE PORTFOLIOS

1. PostgreSQL Declarative Table Partitioning

When audit logs and invoice tables reach 100,000,000 rows, B-tree index depths degrade. PropLedger implements Range Partitioning by Year/Month:

// CODE LISTING: SQL – POSTGRESQL DECLARATIVE PARTITIONING

```
-- Declarative Table Partitioning by Range
CREATE TABLE audit_logs (
    id BIGSERIAL,
    table_name VARCHAR(100) NOT NULL,
    record_id BIGINT NOT NULL,
    action VARCHAR(20) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL,
    old_values JSONB, new_values JSONB
) PARTITION BY RANGE (created_at);

-- Monthly Child Partitions:
CREATE TABLE audit_logs_2026_09 PARTITION OF audit_logs
    FOR VALUES FROM ('2026-09-01') TO ('2026-10-01');

CREATE TABLE audit_logs_2026_10 PARTITION OF audit_logs
    FOR VALUES FROM ('2026-10-01') TO ('2026-11-01');
```

2. Benefits: Partition Pruning & Cold Data Detachment

Queries filtering by date scan only the relevant monthly partition (Partition Pruning). Old partitions (> 7 years) can be detached and archived to cold S3 storage without locking the live database.

8. 360-DEGREE INTERVIEW PITCH FRAMEWORK

HOW TO PRESENT PROPLEDGER TO HIRING MANAGERS AND ARCHITECTS

1. The 2-Minute Executive Pitch

'PropLedger is a production-grade property management and financial operations platform engineered to the standards of Yardi Voyager and RealPage. I built it to solve real enterprise challenges: eliminating unit double-booking using PostgreSQL btree_gist temporal exclusion constraints, managing multi-invoice split payments via a double-entry subledger with row-level trigger consistency, and preventing concurrency race conditions using pessimistic row locks (SELECT FOR UPDATE) and optimistic JPA versioning. On the frontend, I used React 19, TypeScript, and TanStack React Query to implement coordinated multi-cache invalidations across financial dashboards.'

2. Key Themes to Emphasize in Interviews

Interview Domain	What Novice Engineers Say	What You Say (PropLedger Architecture)
Database Design	'I made tables and foreign keys.'	'I normalized to 3NF/BCNF with trigger-managed denormalizations.'
Concurrency	'I used synchronized in Java.'	'I used btree_gist exclusions and ordered pessimistic row locks.'
State Management	'I put everything in Redux.'	'I separated server state (React Query) from client UI state.'
Security	'I used standard JWT.'	'I built a stateless OncePerRequestFilter with BCrypt-12 and RBAC.'

9. SYSTEM DESIGN LEADERSHIP & TRADEOFF SCENARIOS

HANDLING AMBIGUITY, SCALABILITY & ENGINEERING TRADEOFFS

1. Senior Architecture Scenarios

Scenario 1: Defending Monolith vs Microservices in FinTech

Hiring managers often probe: 'Why didn't you build 10 microservices?' Your answer: 'Real estate accounting requires absolute ACID consistency across leases, invoices, and payments. A distributed architecture requires distributed 2-phase commits or eventual consistency sagas, which introduce failure states and latency overhead. A modular monolith with clean domain separation delivers sub-millisecond joins and guaranteed transactional consistency while scaling horizontally via stateless web containers.'

Scenario 2: Handling High-Concurrency Month-End Accounting

Explain how you decoupled OLTP checkouts from heavy analytical reporting by introducing CQRS-Lite read/write splitting with AbstractRoutingDataSource, routing rent rolls to read replicas.

LEADERSHIP PRINCIPLE

Great engineers don't choose technologies because they are trendy; they choose technologies that guarantee data integrity, maintainability, and operational simplicity under load.

10. MASTER TECHNICAL CHEAT SHEET & REPOSITORY INDEX

QUICK REFERENCE: PORTS, ENDPOINTS, COMMANDS & DIRECTORY INDEX

1. Local Execution & Runtime Reference

Service Component	Port / Protocol	Local Access URL	Credentials / Commands
Spring Boot API	Port 8080 (HTTP)	http://localhost:8080	mvn spring-boot:run
Swagger UI	Port 8080 (HTTP)	http://localhost:8080/swagger-ui.html	Interactive OpenAPI 3.0
React 19 Frontend	Port 5173 (HTTP)	http://localhost:5173	npm run dev
PostgreSQL 16	Port 5432 (TCP)	localhost:5432/propledger	postgres / postgres
Manager Account	JWT Auth	POST /api/auth/login	manager@propledger.com / Password123!
Super Admin	JWT Auth	POST /api/auth/login	admin@propledger.com / Password123!

2. Complete 10-Volume PDF Library Index

Volume	Document Title	Primary Focus
Vol 01	Enterprise System Architecture	Multi-tier design, HikariCP, CQRS, and infrastructure
Vol 02	Relational Schema & Data Modeling	PostgreSQL schema, entities, and temporal models
Vol 03	Advanced SQL & Analytics Engine	CTEs, Window Functions, Rent Roll, Aging AR, P&L
Vol 04	Normalization & Indexing Strategy	1NF-BCNF proofs, ESR rule, and EXPLAIN ANALYZE
Vol 05	Transactions & Concurrency Control	ACID boundaries, row locking, and deadlocks
Vol 06	Identity, JWT & Spring Security	Stateless auth, BCrypt, RBAC, and OncePerRequestFilter
Vol 07	Spring Boot API & Domain Services	DTOs, builder mappers, validation, and RFC 7807
Vol 08	Financial Subledger & Billing Engine	Batch billing, payment allocations, and OpEx/CapEx
Vol 09	Frontend Architecture & React 19	TypeScript, TanStack Query, Tailwind, and Recharts
Vol 10	FullStack Integration & Interview Mastery	System integration, testing, and interview mastery